

LAB 02: TÍNH LIÊN THÔNG CỦA ĐỒ THỊ

MỤC TIÊU

Hoàn tất bài thực hành này, sinh viên có thể:

- Duyệt đồ thị bằng BFS, DFS.
- Kiểm tra đồ thị liên thông.
- Đếm số thành phần liên thông của đồ thị.
- Áp dụng vào một số bài toán thực tế.

Bài tập áp dụng: Tìm miền liên thông trong ảnh, Đường hầm bí mật, Bản đồ các vùng đảo... Thời gian thực hành: từ 120 phút đến 400 phút

TÓM TẮT

- 2 cách duyệt đồ thị: DFS, BFS.

Chương trình chính	DFS	BFS
begin for $v \in V$ do chưaXét [v] = true; for $v \in V$ do if chưaXét [v] then DFS(v); // BFS(v) end	procedure DFS(v) begin ThămĐỉnh(v); chưaXét [v] = false; for $u \in \text{Kề}(v)$ do if chưaXét [u] then DFS(u); end	procedure BFS(v) begin QUEUE = $\emptyset$; QUEUE $\leftarrow v$; // Kết nạp v vào QUEUE chưaXét [v] = false; while QUEUE $\neq \emptyset$ do $p \leftarrow$ QUEUE; ThămĐỉnh(p); for $u \in \text{Kề}(p)$ do if chưaXét [u] then QUEUE $\leftarrow u$; chưaXét [u] = false; end;

- Một đồ thị vô hướng được gọi là liên thông nếu có đường đi giữa mọi cặp đỉnh phân biệt của đồ thị.
- Thành phần liên thông (TPLT) của G là đồ thị con liên thông tối đại của G .
- Đồ thị có hướng gọi là liên thông mạnh nếu có đường đi từ a tới b và từ b tới a với mọi cặp đỉnh a và b của đồ thị.
- Đồ thị có hướng gọi là liên thông yếu nếu có đường đi giữa 2 đỉnh bất kỳ của đồ thị vô hướng nền.

NỘI DUNG THỰC HÀNH

Cơ bản

Sinh viên đọc kỹ phát biểu bài tập và thực hiện theo hướng dẫn:

Sử dụng lại lớp GRAPH thực hành ở tuần trước, cho trước một đồ thị vô hướng, sinh viên kiểm tra đồ thị có liên thông không và nếu không thì có bao nhiêu thành phần liên thông, cho biết cụ thể các đỉnh trong từng thành phần liên thông

Phân tích

- Trong thuật toán xác định liên thông, ta cần đánh nhãn các TPLT và đếm số TPLT, vì vậy trong thuộc tính của lớp GRAPH cần phải có:

```
int arrnLabel[MAX]; // nhãn cho các đỉnh, 0 là chưa duyệt
//1, 2, ... là đã duyệt và đỉnh thuộc miền liên thông tương ứng
int nSoMienLienThong;
```

- Các thao tác cần thực hiện: Duyệt đồ thị, Xét liên thông, In các thành phần liên thông.

Chương trình mẫu

```
int GRAPH::XetLienThong(void)
{
    for (int i=0; i<n; i++) //Thao tác này dùng để làm gì?
        arrnLabel[i] = 0;

    nSoMienLienThong = 0;

    for (int i=0; i<n; i++){
        if (arrnLabel[i]==0){ //Điều kiện này có ý nghĩa gì?
            nSoMienLienThong++; //Thao tác này dùng để làm gì?
            Visit(i, nSoMienLienThong);
        }
    }

    return nSoMienLienThong;
}

void GRAPH::InThanhPhanLienThong(void)
{
    if(nSoMienLienThong == 1){
        printf("Do thi lien thong.\n");
    }
    else{
        printf("Do thi khong lien thong, co %d TPLT.\n", nSoMienLienThong);

        for (int i=1; i<=nSoMienLienThong; i++){
            printf("Mien lien thong #%d:", i);

            for (int j=0; j<n; j++){
                if (arrnLabel[j]==i){ //Điều kiện này có ý nghĩa gì?
                    printf(" %d", j);
                }
            }

            printf("\n");
        }
    }
}

void GRAPH::Visit(int i, int nLabel)
{
    arrnLabel[i] = nLabel;

    for (int j=0; j<n; j++){
        if((arrnLabel[j]==0)&&(a[i][j]||a[j][i])){ //Điều kiện này có ý nghĩa gì?
            Visit(j, nLabel); //Thao tác này dùng để làm gì?
        }
    }
}
```

Yêu cầu

1. Bổ sung và biên dịch lại chương trình ở bài tập tuần trước.
2. Chạy chương trình với các dữ liệu sau

5	5	7
0 2 0 4 0	0 1 0 0 1	0 0 1 0 0 0 1
2 0 -3 1 2	1 0 1 0 0	0 0 0 1 0 0 0
0 -3 0 2 2	0 1 0 0 1	1 0 0 0 0 0 0
4 1 2 0 0	0 0 0 0 1	0 1 0 0 0 1 0
0 2 2 0 0	1 0 1 1 0	0 0 0 0 0 0 1
		0 0 0 1 0 0 0
		1 0 0 0 1 0 0

3. Giải thích ý nghĩa các hàm trong code mẫu và chi tiết từng dòng lệnh.
4. Nếu ta thay đổi code trong hàm XetLienThong thành

```
for (int i=0; i<n; i++){
    if (arrnLabel[i]){
        nSoMienLienThong++;
        Visit(i, nSoMienLienThong);
    }
}
```

thì kết quả có gì thay đổi, giải thích lý do.

5. Nếu ta thay đổi code trong hàm XetLienThong thành

```
for (int i=0; i<n; i++){
    if (arrnLabel[i]==0){
        Visit(i, nSoMienLienThong);
        nSoMienLienThong++;
    }
}
```

thì kết quả có gì thay đổi, giải thích lý do.

6. Nếu ta thay đổi code trong hàm XetLienThong thành

```
for (int i=0; i<n; i++){
    nSoMienLienThong++;

    if (arrnLabel[i]==0){
        Visit(i, nSoMienLienThong);
    }
}
```

thì kết quả có gì thay đổi, giải thích lý do.

Áp dụng – Nâng cao

1. Viết hàm DFS duyệt đồ thị theo chiều sâu để in ra các đỉnh.
2. Viết hàm BFS duyệt đồ thị theo chiều rộng để in ra các đỉnh.
3. Thủ tục Visit đã cho duyệt đồ thị theo chiều rộng hay chiều sâu? Thay đổi hàm Visit cho cách duyệt còn lại.

H.D: Sử dụng lại cấu trúc *QUEUE* đã cài đặt trong môn CTDL.

4. Nếu dữ liệu tăng lên thì hàm Visit có thể xảy ra hiện tượng gì? Giải quyết sự cố trên.

H.D: Sử dụng lại cấu trúc *STACK* đã cài đặt trong môn CTDL.

BÀI TẬP THÊM

1. Xây dựng các hàm xác định liên thông mạnh và yếu trong đồ thị có hướng,
2. Bài toán “Đường hầm bí mật”

Trong các lâu đài cổ ở châu Âu người ta thường xây dựng các đường hầm bí mật để thoát hiểm trong các trường hợp khẩn cấp. Các đường hầm chỉ có thể vào từ một cửa vào duy nhất tại phòng Trung tâm và thoát ra ở rất nhiều cửa ra. Các cửa ra đều nằm ở rìa lâu đài, do vậy, nếu thoát ra được rìa lâu đài thì coi như đã thoát hiểm. Để nguy trang, người ta cho đào nhiều nhánh hầm cụt và cửa vào giả. Ngoài ra, để tăng khả năng thoát hiểm, người ta còn xây dựng các đường hầm giao nhau tại một số vị trí. Để nghiệm thu công trình, chủ lâu đài cần kiểm tra xem từ phòng trung tâm có thể thoát hiểm qua hệ thống đường hầm hay không. Hãy lập trình giúp chủ lâu đài kiểm tra hệ thống trên. Biết rằng lâu đài là một hình vuông được chia lưới ô vuông gồm n dòng, n cột. Trên hoạ đồ, ô ở dòng i cột j được ghi số 1 nếu có đường hầm, số 0 nếu không có (ô ở góc trên trái có toạ độ $(0, 0)$). 2 ô chỉ có thể thông nhau nếu chúng có chung cạnh.

Dữ liệu nhập vào từ tập tin văn bản “*duongham.in*” gồm:

- Dòng đầu chứa 3 số nguyên dương n (≤ 50), D , C (D , C là dòng và cột của phòng trung tâm).
- n dòng tiếp theo, mỗi dòng chứa n số là các số ở các vị trí tương ứng trên hoạ đồ.
- Ghi chú: các số trên cùng dòng cách nhau bởi khoảng trắng.

Kết quả tìm được ghi ra tập tin văn bản “*duongham.out*”. Dòng đầu chứa số m là số ô phải đi qua, nếu không thoát được thì $m = -1$. Trong trường hợp thoát được, m dòng tiếp theo, m dòng tiếp theo: mỗi dòng chứa 2 số là số hiệu dòng cột của các ô phải đi qua theo đúng trình tự của một cách thoát hiểm.

Ví dụ:

DUONGHAM.IN	DUONGHAM.OUT
4 2 1	3
0 1 1 0	2 1
1 0 0 1	2 2
1 1 1 1	3 2
0 1 1 0	

3. Bài toán “Bản đồ các vùng đảo”

Lân cận 4 và lân cận 8

Khi làm việc với các dạng lưới kẻ ô như bàn cờ, bản đồ... thì người ta đưa ra khái niệm 2 ô kề nhau. Có nhiều loại kề nhau như kề nhau theo bước đi con mã, theo đường chéo, theo lân cận 4, lân cận 8... ở đây chúng ta sẽ tìm hiểu về kề nhau theo lân cận 4 và lân cận 8.

- Hai ô bất kỳ A và B có toạ độ lần lượt là (i_A, j_A) và (i_B, j_B) (i là dòng, j là cột) được gọi là kề nhau theo lân cận 4 khi và chỉ khi $|i_A - i_B| + |j_A - j_B| = 1$ (tức là 4 ô xung quanh trên dưới trái phải là lân cận của ô trung tâm).
- Hai ô bất kỳ $A(i_A, j_A)$ và $B(i_B, j_B)$ được gọi là kề nhau theo lân cận 8 khi và chỉ khi $|i_A - i_B| \leq 1$ và $|j_A - j_B| \leq 1$ (tức là 8 ô xung quanh là lân cận của ô trung tâm).

- Các ô có thể đi đến được với nhau theo lân cận 4 được gọi tắt là một miền liên thông 4. Các ô có thể đi đến được với nhau theo lân cận 8 được gọi tắt là miền liên thông 8.

Ví dụ:

									1	2		
										3		
											4	
	Lân cận 4			Lân cận 8								

- ✓ Các ô màu xám lần lượt là những ô kề nhau theo lân cận 4 và 8 với ô trung tâm (màu đen).
- ✓ Ô 1 và ô 2 (ô màu xanh) kề nhau theo lân cận 4 (và dĩ nhiên theo lân cận 8 luôn). Tương tự ô 2 và ô 3 cũng kề nhau theo lân cận 4. Ô 3 và ô 4 chỉ kề nhau theo lân cận 8, không theo lân cận 4. Ô 1 và ô 3 cũng kề nhau theo lân cận 8.
- ✓ Các ô 1, 2 và 3 tạo thành một miền liên thông 4 vì từ 1 đi được đến 2, từ 2 đi được đến 3 theo lân cận 4 (từ 3 không đến 4 được theo lân cận 4). Ô số 4 bản thân nó là một miền liên thông 4.
- ✓ Nếu xét theo liên thông 8 thì cả 4 ô (1, 2, 3, và 4) tạo thành một miền liên thông 8.

Yêu cầu

Khi khảo sát các đảo ở một vùng biển nọ, người ta ghi kết quả khảo sát lại thành một bản đồ nhị phân trong đó số 0 cho biết biển và số 1 là đất liền. Một đảo chính là một miền liên thông 4 các số 1 trên bản đồ. Từ một bản đồ cho trước, hãy đếm số lượng đảo của vùng biển.

Dữ liệu ban đầu được cho vào từ file văn bản *bando.txt* có cấu trúc như sau:

- Dòng đầu tiên chứa 2 số nguyên dương M và N ($M, N \leq 1000$) cho biết số dòng (M) và số cột (N) của bản đồ.
- M dòng tiếp theo, mỗi dòng chứa N số nguyên dương (0 hoặc 1) ứng với các vị trí là biển hoặc đất liền.
- Các số trên cùng một dòng cách nhau bởi ít nhất một khoảng trắng hoặc ký tự tab.

Hãy viết chương trình và xuất ra file: dao.txt chứa một số nguyên dương duy nhất cho biết số đảo tìm thấy được trên bản đồ.

Ví dụ:

bando.txt	dao.txt
4 8 0 1 0 0 1 0 0 0 1 1 1 0 1 1 0 0 0 1 0 0 0 0 1 0 0 0 0 0 1 1 1 0	3

4. Cho một ảnh bitmap, xác định số miền liên thông có trong ảnh theo lân cận 4 và 8.